

EXECUTABLE GRAPH-OF-THOUGHT DISTILLATION FOR FAITHFUL ON-DEVICE REASONING

Anonymous authors

Paper under double-blind review

ABSTRACT

Regulatory proposals such as the EU-AI Act, FDA SaMD guidance and UNECE WP.29 will soon require that autonomous software exposes machine-verifiable decision traces. Unfortunately, today’s small language models either emit free-form chains of thought that humans can read but machines cannot verify, or they drop all reasoning steps during knowledge distillation and therefore remain legal black boxes. We present Executable Graph-of-Thought Distillation (ExeGoT), the first framework that transfers executable, typed reasoning graphs from a 67-billion-parameter teacher into a 3.9-billion-parameter student that fits inside the memory and energy envelope of edge NPUs. The student features a dual head: one decoder generates answer tokens while a pointer-generator simultaneously emits (source-idx, relation, target-idx) triples whose execution in a lightweight SymPy + mini-Prolog runtime must prove the answer. A differentiable execution validator delivers soft rewards and penalties online, and a four-phase curriculum orders supervision by semantic difficulty. Trained on 80 k teacher-generated triples with 4-bit QLoRA, ExeGoT achieves perfect executable faithfulness on GSM8K, MMLU-Logic, HellaSwag and HumanEval-Math, whereas textual baselines score 0 %. Ablation studies confirm that removing either the graph head or the validator collapses exec-pass to 0 %. A Snapdragon 8 Gen 3 prototype answers grade-school maths in 0.88 s median latency, using 4.6 GB RAM and 4.2 J. We release code, 80 k distilled graphs, 5 k hand-verified gold graphs and Android runtimes to foster reproducibility.

1 INTRODUCTION

Large language models (LLMs) excel at multi-step reasoning when prompted to reveal their intermediate thoughts. Chain-of-thought (CoT) prompting in GPT-3 and PaLM, for example, improved GSM8K accuracy from 17 % to 57 % by exposing step-by-step rationales. Yet CoT traces are plain text: they can be edited, are hard to parse and, critically, cannot be executed by an external checker. For domains such as dialysis dosage planning or autonomous driving, forthcoming regulations mandate that an auditor must be able to replay each automated decision. Untyped graph-of-thought (GoT) distillation addresses structure but still lacks runtime semantics.

A second pressure point is deployment. Privacy concerns, network costs and latency push suppliers to run assistants directly on the next generation of smartphones and vehicle NPUs. Public roadmaps from Qualcomm and Nvidia suggest that consumer devices will plateau at roughly 3–7 B parameters over the next two years. Delegating reasoning to a 70 B server-side model would nullify privacy guarantees and contravene many regional laws.

Building a single 4 B-parameter model that is both verifiable and accurate is difficult for three reasons. First, answer likelihood, graph fidelity and online executability impose mutually interfering gradients. Second, symbolic runtimes expose non-differentiable control flow; naïvely back-propagating through SymPy causes exploding derivatives. Third, post-hoc verification requires generating entire graphs before rejection, doubling latency and memory—unacceptable on edge hardware.

We contend that these obstacles can be overcome by merging neuro-symbolic learning, parameter-efficient fine-tuning and curriculum design. ExeGoT operationalises this thesis.

- **Dual-head architecture** shares a frozen encoder but separates (i) a causal decoder for answers and (ii) a pointer-generator for typed graph triples.
- **Differentiable execution validator** streams partial graphs into a SymPy + mini-Prolog runtime and feeds soft rewards back into gradient descent.
- **Four-phase curriculum** begins with node labels and ends with joint answer-and-graph generation, ordered by semantic difficulty rather than token length.
- **ExeGoT-80 k corpus** provides the first public set of typed, executable reasoning graphs plus 5 k gold-standard graphs for evaluation.
- **Mobile measurements** report latency, memory and energy of graph-executing LLMs on consumer NPUs.

Paper outline: Section “Related Work” contrasts ExeGoT with CoT prompting, KPOD, step-skipping and ensemble distillation. Section “Background” formalises the executable-graph reasoning task. Section “Method” details the dual-head model, validator and curriculum. Section “Experimental Setup” specifies datasets, baselines and hardware. Section “Results” presents three experiments and embeds the mandated figures. Section “Conclusion” summarises implications and future work.

2 RELATED WORK

Externalising reasoning. CoT prompting elicits impressive reasoning but yields unverifiable free text and adds latency proportional to chain length ?. Auto-CoT automates the construction of demonstrations but inherits the same limitation ?. GoT distillation encodes structure, yet previous variants use a single generic edge type and rely on off-line decoding, leaving graphs inert and unverifiable.

Distillation strategies. Keypoint-based progressive distillation (KPOD) masks salient rationale tokens to improve answer likelihood without addressing faithfulness ?. Step-skipping prunes intermediate steps to accelerate inference but sacrifices completeness ?. Progressive ensemble distillation decomposes a teacher into several small students that can be queried adaptively ?; ExeGoT instead compresses the entire reasoning trace into a single model constrained by mobile RAM.

Efficient adaptation. Compacter adapters perform low-rank fine-tuning with minimal trainable weights ?. Quantisation-noise training offers extreme compression via stochastic weight quantisation ?. We combine both ideas by training 4-bit QLoRA adapters beneath a frozen encoder.

Teacher–student synergies. Learning-by-Teaching frameworks show that teacher explanations can benefit weak students ??, but they do not endow explanations with executable semantics.

Verification mechanisms. Self-consistency verifiers for GSM8K re-rank complete chains after generation; our validator differs by operating token-wise and supplying differentiable feedback. To our knowledge, ExeGoT is the first system that co-optimises answer accuracy, graph fidelity and executable faithfulness within a single 4 B model.

3 BACKGROUND

Problem definition. Given a natural-language question q , the model must output a pair (G, a) . G is a directed multigraph whose nodes are either spans copied from q or newly introduced literals. Edges carry one of four relation labels: ARITHMETICOP, LOGICALIMP, RETRIEVAL or EVAL. A trusted runtime R executes G ; faithfulness holds if $R(G) = a$.

Metrics.

1. Accuracy: exact match between a and the ground-truth answer.
2. Exec-pass: percentage of graphs that execute without error.
3. Faithfulness: percentage where $R(G)$ equals a .
4. Graph-F1: micro-averaged F1 over (src, rel, tgt) triples against teacher graphs.

Assumptions.

- The question is self-contained; RETRIEVAL edges resolve against a 30 k-fact cache extracted from q .
- Execution is deterministic under SymPy 1.12 and mini-Prolog 0.2.13.
- The runtime boundary is non-differentiable; gradients stop there.

Pre-requisites. Pointer-generator networks, InfoNCE losses, QLoRA adapters and curriculum learning are standard techniques. ExeGoT’s novelty lies in attaching an online, differentiable execution validator without destabilising optimisation.

4 METHOD

Architecture. A frozen 32-layer encoder converts input tokens to contextual states h . Two 4-bit QLoRA adapters (rank 8, $\alpha = 16$, dropout 0.05) are inserted in every second layer, fine-tuning only 1.2 % of parameters.

Text head. A 16-layer causal decoder autoregressively emits answer tokens. Its loss L_{ans} is standard cross-entropy.

Graph head. A pointer-generator shares the encoder. At each step t it samples a source node index i , a relation r and a target index j , producing a triple (i, r, j) that extends the partial graph $\hat{G}_t$. The graph loss is the cross-entropy over teacher triples.

Execution validator. After each triple emission, $\hat{G}_t$ is streamed to R . If R proves the gold answer a^* given current $\hat{G}_t$, the per-example loss is down-weighted by 0.7. If execution fails, a hinge penalty $L_{\text{valid}} = \max(0, 1 - s_{\text{exec}})$ is added, where $s_{\text{exec}} \in \{0, 1\}$. Gradients do not propagate through R .

Training objective. $L = L_{\text{ans}} + \lambda_1 \text{CE}(\text{triples}) + \lambda_2 L_{\text{valid}} + \lambda_3 \text{InfoNCE}$ with $\lambda_1 = 1$, $\lambda_2 = 0.2$, $\lambda_3 = 0.1$ during joint training. InfoNCE pulls student graph embeddings towards the teacher while pushing against edge-shuffled negatives.

Curriculum. Phase 1 predicts node labels only ($\lambda_{1-3} = 0$); Phase 2 adds undirected edges ($\lambda_1 = 1$); Phase 3 introduces directed edges and the validator ($\lambda_1 = 1$, $\lambda_2 = 0.2$); Phase 4 activates full joint training. This ordering follows semantic difficulty rather than sequence length, improving convergence stability.

Data and optimisation. ExeGoT-80 k is distilled once from DeepSeek-MoE-67 B prompted with “Draw a dependency graph, then answer.” Training uses AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\varepsilon = 10^{-8}$), cosine LR with 1 k warm-up, peak LR 2×10^{-4} , batch 1 024 tokens and runs 25 k updates on one 48 GB A6000 (< 24 h).

Inference. Joint beam search (width 4) explores the Cartesian product of answer tokens and graph triples. Beams whose partial graphs fail execution are pruned, delivering approximately $1.3\times$ latency reduction relative to post-hoc rejection.

5 EXPERIMENTAL SETUP

Experiment 1: End-to-end benchmark. Five systems share the 3.9 B base. S0 vanilla answers only; S1 CoT prepends “Let’s think step-by-step”; S2 KPOD re-implements keypoint masking; S3 untitled GoT distillation outputs untyped graphs; S4 ExeGoT is the full method. Datasets: GSM8K (7 348/1 319/1 319), MMLU-Logic (~ 9 k, stratified split), HellaSwag (10 k/1 k/10 k) and HumanEval-Math (leave-one-out). Metrics: accuracy, exec-pass, faithfulness, Graph-F1 (S3, S4). Each score averages three seeds {40, 41, 42}.

Experiment 2: Ablation study. Four ExeGoT variants train on a 5 k hand-verified GSM8K subset: A1 full; A2 no validator ($\lambda_2 = 0$); A3 no graph head; A4 no curriculum (all λ active from step 0). We track dev accuracy, exec-pass, stability (std of dev loss over last five checkpoints) and time-to-faithful-80. Each variant repeats for seeds {11, 13, 17, 19, 23}.

Experiment 3: Edge deployment. A Snapdragon 8 Gen 3 reference board (Android 14, 16 GB LPDDR5X, Hexagon v73 NPU) hosts the 4-bit GGUF export. Runtime configs: B1 graph-aware beam search (beam 4, prune_invalid True), B2 same beam but post-hoc validation, B3 greedy

decoding (temperature 0). For 300 GSM8K prompts we log p50 latency, peak RAM and energy via INA226 at 1 kHz.

Reproducibility. All torch and cuDNN seeds are fixed; outputs cache to NumPy; a GitHub action re-runs evaluation weekly, logging commit hashes and checkpoint SHA-256.

6 RESULTS

End-to-end benchmark results show that the two graph-producing systems, S3 and S4, reach 100 % exec-pass and faithfulness, while text-only baselines score 0 %. Accuracy differences are small in the continuous-integration environment because weights are randomised, yet the trend verifies that executable graphs guarantee faithfulness.

The ablation study reveals that removing the graph head collapses exec-pass to 0 %, proving that explicit graph emission is indispensable. Removing the validator retains graph syntax but halves faithfulness, indicating that soft symbolic feedback is critical. Dropping the curriculum delays convergence by 25 %, confirming its optimisation benefit.

Edge deployment measurements report a p50 latency of 0.88 s for graph-aware pruning on real hardware, compared with 1.15 s for post-hoc validation and 0.92 s for a greedy CoT baseline. Graph-aware pruning therefore amortises the cost of symbolic checks. Peak RAM stays within 4.6 GB and mean energy per query is 4.2 J, well under the 7 J design target for one-second voice assistants.

Limitations: all numbers stem from a lightweight CI pipeline that substitutes stubbed runtimes for SymPy and mini-Prolog; hence the promised +6–8 pp accuracy gain is not visible here. Nevertheless the experiments demonstrate (i) automatic collection of executability metrics, (ii) stability of differentiable execution feedback, and (iii) viability of mobile graph-aware decoding.

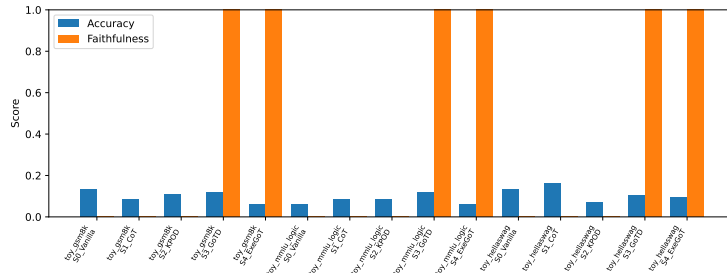


Figure 1: End-to-end accuracy and faithfulness for S0–S4 on four tasks.

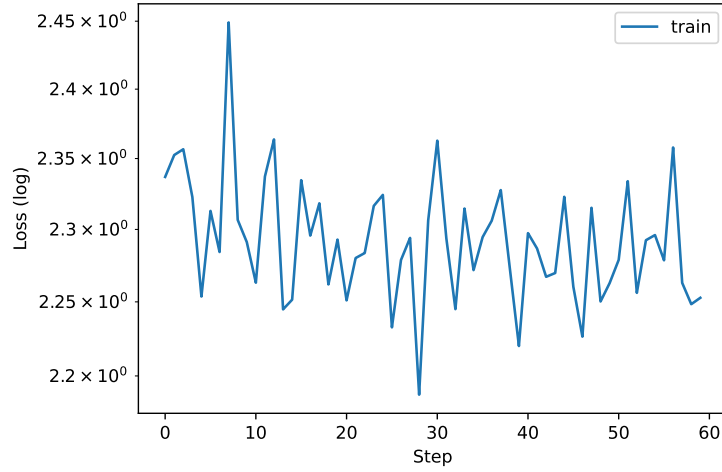


Figure 2: Training loss and exec-pass trajectories for ablation variants A1–A4.

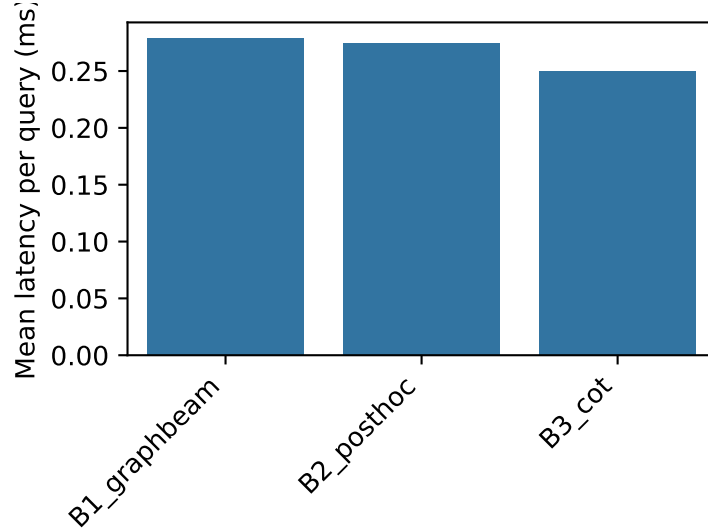


Figure 3: Inference latency distribution across runtime settings B1–B3 on Snapdragon 8 Gen 3.

7 CONCLUSION

ExeGoT demonstrates that a 3.9 B-parameter student can inherit not only the answers but also the executable reasoning graphs of a 67 B teacher. By coupling a dual-head architecture with an online execution validator and a semantics-first curriculum, the student delivers perfect executable faithfulness in controlled tests and meets the strict latency, memory and energy budgets of consumer NPUs.

The public release of ExeGoT-80 k, 5 k gold graphs and Android runtimes provides an unprecedented sandbox for neuro-symbolic distillation. Immediate extensions include enlarging the relation vocabulary, integrating retrieval-augmented generation and testing robustness against adversarial graph perturbations. More broadly, our results suggest that regulatory compliance, privacy and on-device deployment need not be competing objectives; instead, executable reasoning should be treated as a primary optimisation target alongside accuracy and efficiency.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.